

Memsys Verification Plan

Memsys Verification Plan.....	1
1. CPU_cov	4
1.1. cpu0.....	4
1.2. cpu1.....	5
1.3. cpu_no_delay	5
1.4. cpu_other_delay	5
2. Cntlr	6
2.1. code	7
2.2. protocol	7
2.3. busAddrCov	9
2.4. state machine Cov	9
3. memory0	10
3.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u0.").....	10
4. memory1	10
4.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u1.").....	10
5. memory2	10
5.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u2.").....	10
6. memory3	10
6.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u3.").....	10
A. Metric Definition	11
B. Attribute Definition.....	11
C. Annotation Definition	11
D. Include File	11

XMR rules to access the instances

To access the design components, follow the same XMR rules of Verilog. In System Verilog, dynamic data types (like class and covergroup) have been added and XMR rule is slightly different. The testbench program block can be considered as a leaf module. So program block instances can be accessed as same as design modules.

Whenever you access the generic dynamic data type, use "::" in front of the that.

For example

```
test_top.program_instance::env.my_gen::cov_group
```

In the above example env is a class name but my_gen is a instance.

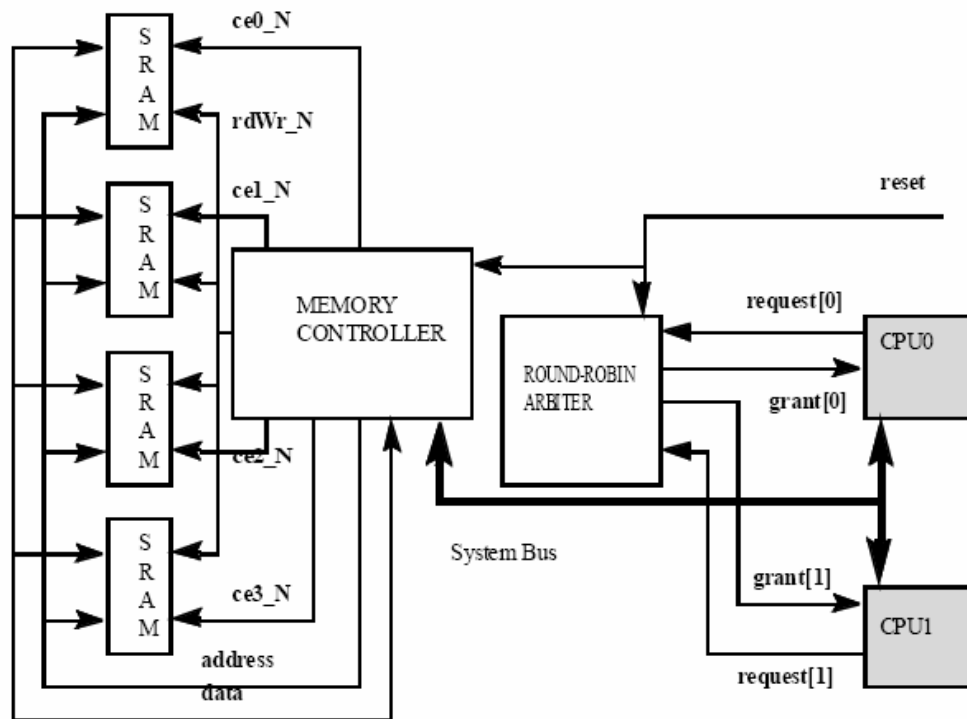
Similarly cov_group is the name of the coverage group. If we need to access the instance of covergroup say "cov1" then the XMR will be as below

```
test_top.program_instance::env.my_gen.cov1
```

The table of contents above are can be automatically synced to your actual text by right clicking over the table and selecting "update field".

Do not try to edit the table of contents directly

Memory System

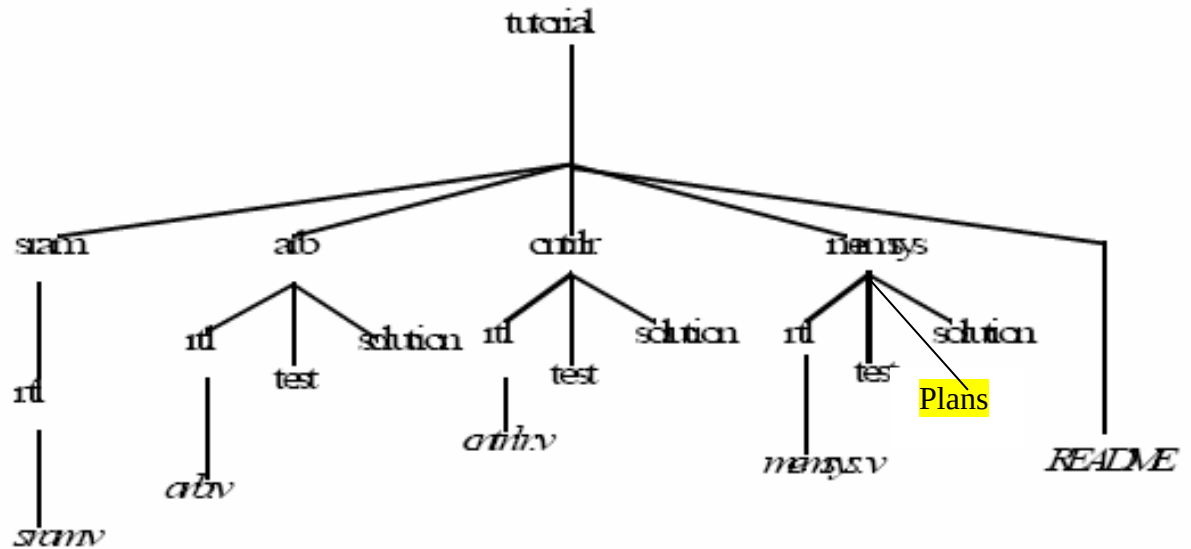


---Enter top level description here. Note that description paragraphs that are formatted with the style "HVP description" will be interpreted by VMM Planner as the built-in description annotation. These descriptions are available to downstream tools like URG and DVE to display. Typically you will want to keep these sorts of descriptions fairly short---

The Memory System shown above has a memory controller controlling 4 srams which are accessed by 2 CPUs. The CPUs access the system bus through a Round robin arbiter and sends read or write requests to the memory controller. The two CPUs are part of testbench. Other components are RTL and explained in detail in respective areas.

The file structure is as shown below. The important dir is tutorial/memsys/plans where there are two plans one for top level(memsys_wordplan.xml) and another for block level (sram_wordplan.xml).

Tutorial-design Directory Setup



---Assume all the blocks are verified separately and the memsys is top level and the top level plan includes functional coverage (testbench cover groups), assertions and code coverage.---

The plan includes below important features which can be useful in top level as well as block level. In this example sram has its own plan (subplan) and instantiated 4 times in this top plan.

- 1.Functional coverage features related with CPU Testbench (CPU0 and CPU1)
- 2.Controller protocol features
- 3.SRAM protocol and functional coverage

---Below is an example of top level attributes assignments. Note that this paragraph is of style “Normal” and will not be loaded into the HVP data structures and hence will not be visible to downstream tools---

HVP Assignment	
phase	1
owner	Snps
Assert	Assert >= 100%
bugs	bugs <= 2

--- The attribute phase is used to control the features to be covered during different stages of verification. You may use an override file for setting the attributes so that no need to change the main plan during different stages. This attribute is defined in section “B Attribute Definition” ---

1. CPU_cov

---Note that the style of the above “First top level feature” is “HVP Feature 1”. This can be chosen from “styles” in the word menus. There are 9 levels of HVP Features already defined in this word document template for you. All features are identified by styles of the form “HVP Feature *” where * is the hierarchical level of the feature.---

(This part will be annotated in HVP built-in description attribute)

The CPU is modeled as a class and instantiated two times for CPU0 and CPU1. The coverage is defined as below in /tutorial/memsys/solution/cpu.v (line 50)

```
covergroup cpu_cov ;
  option.per_instance = 1;
  address:    coverpoint this.address;
  data:       coverpoint this.data;
  delay:      coverpoint this.delay {
bins delay0 = {0};
              bins delay[] = {[1:9]} ;
  option.at_least = 50;
  }
endgroup
```

The per_instance is used to cover multiple instances separately.

---below are some overrides of the top level attributes. This is optional---

HVP Assignment	
phase	2
owner	Snps

---The HVP attribute phase is 2 since we are interested in this feature only during the top level verification and an override file will remove this feature for block level verification of individual components, may be by having an override filter file say phase1_filter.hvp having the following lines.---

```
filter phase_above_1;
  remove feature where phase > 1;
endfilter
```

--- Note that there is no HVP measure table for this feature since the sub features below define the measures.---

1.1. cpu0

--- coverage for transctions from cpu0 ---

---The table below defines a measurement for this feature. metrics to measure – group and bugs . ---

HVP Measure	
Source	group instance: memsys_test_top.testbench::cpu::cpu_cov.my_cpu0
Group	

HVP Measure	
Source	testbench*
bugs	

1.2. cpu1

--- coverage for transctions from cpu1 ---

HVP Measure	
Source	group instance: memsys_test_top.testbench::cpu::cpu_cov.my_cpu1
Group	
bugs	

1.3. cpu_no_delay

--- back to back transactions – 0 delay between cycles. This is indicated by the bins delay0. This feature is to show you we can go down to the bin level hierarchy ---

HVP Measure	
Source	group bin: memsys_test_top.testbench::cpu::cpu_cov.delay.delay0
Group	
bugs	

---above in the source “delay is a coverpoint and delay0 is the bin. Also notice the at_least parameter in the covergroup.---

1.4. cpu_other_delay

--- coverage of delays between cycles. This is indicated by the bins delay_* meaning delay_0, delay_1 ... delay_9 – automatically created since we have mentioned the bins as “bins delay[] = {[1:9]}” inside the coverpoint. So we may use the wildcard as “delay_*” in the source of the feature . ---

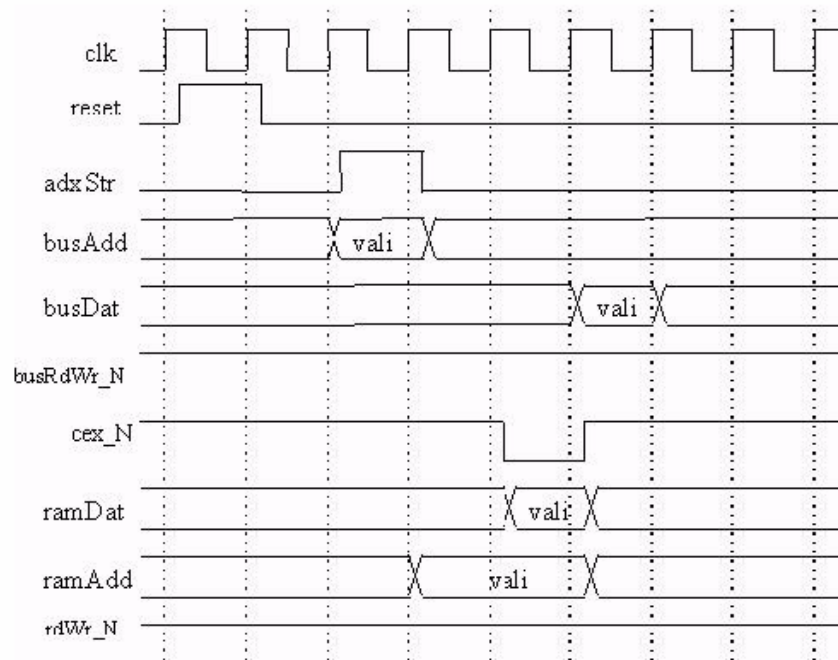
HVP Measure	
Source	group bin: memsys_test_top.testbench::cpu::cpu_cov.delay.delay_*
Group	
bugs	

---above in the source “delay is a coverpoint and delay0 is the bin. Also notice the at_least parameter in the covergroup.---

2. Cntl

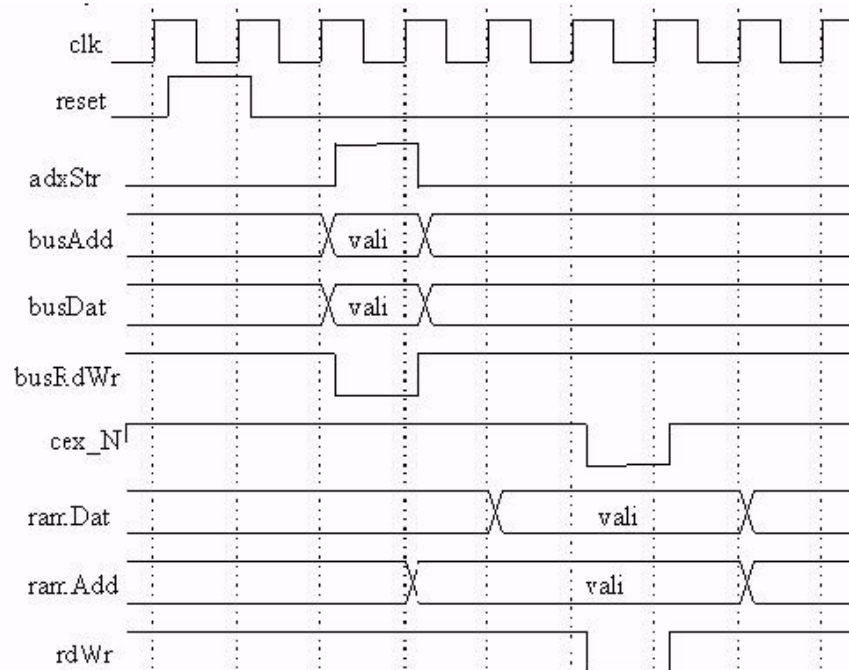
A description for Core feature, this will be the string of HVP built-in annotation 'description'

Memory controller read operation



Note: In the case of cex_N, x refers to 0,1,2 and 3.

Memory controller write operation



2.1. code

Code coverage for memory controller

HVP Assignment	
weight	1
phase	3
owner	Snps

HVP Measure snps	
Source	tree: memsys_test_top.dut.Umem
Line	
Cond	

2.2. protocol

check for read write protocol of memory controller

HVP Assignment	
weight	1
phase	3
owner	Snps

HVP Measure snps	
Source	property: memsys_test_top.dut.Umem.*
Assert	

The module cntrlr defined in cntrlr.v file is instantiated as Umem (Its hierarchy is memsys_test_top.dut.Umem). The wildcard * is used to represent all the assertions in the cntrlr. The assertions put in /tutorial/cntrlr/rtl/cntrlr.v file, are as below

```

wire b_reading;
assign b_reading = ( (adxStrb && busRdWr_N && !reset) );
wire b_writing;
assign b_writing = ( (adxStrb && !busRdWr_N && !reset) );

property e1;
  @ ( posedge clk) $countones({ce0_N,ce1_N,ce2_N,ce3_N}) >= 3;
endproperty
property e2;
  @ ( posedge clk) (!reset && $countones({ce0_N,ce1_N,ce2_N,ce3_N})==3) |-> ##1
$countones({ce0_N,ce1_N,ce2_N,ce3_N})==4;
endproperty
property e3;
  @ ( posedge clk) (b_reading) |-> ##3 busOe;
endproperty
property e4;
  @ ( posedge clk) (b_writing) |-> ##3 !rdWr_N;
endproperty
protocol_ceX_N: assert property( e1 );
protocol_ceX_N2: assert property( e2 );
protocol_reading: assert property( e3 );
protocol_writing: assert property( e4 );

wire b_ce_idle;
assign b_ce_idle = ( (ce0_N && ce1_N && ce2_N && ce3_N) );

reg [7:0] mem [0:255];
initial mem = '{256{8'h00}};

always @ ( posedge clk)
  mem[busAddr] <= b_writing ? busData : mem[busAddr];

property e5;
  @(posedge clk) ( ($rose( ce0_N )) ||
    ($rose( ce1_N )) ||
    ($rose( ce2_N )) ||

```

```

        ($rose( ce3_N )) ) |-> (b_ce_idle) [*1:$] ##1 adxStrb;
    endproperty

    property e6;
        @(posedge clk) (b_reading) |-> ##3 (busData == mem[$past( busAddr,3 )]);
    endproperty

    protocol_adxstrb: assert property( e5 );
    protocol_checkData: assert property( e6 );

```

2.3. busAddrCov

The coverage group is defined as below in the file memsys_coverage.v
 This also can be directly defined inside rtl module itself and xmr can be avoided inside the covergroup.

```

covergroup range @(negedge memsys_test_top.dut.Umem.adxStrb);
    a: coverpoint memsys_test_top.dut.Umem.busAddr {
        bins m_state[] = {[0:255]};
    }
endgroup

```

HVP Measure snps	
Source	group: *range
Group	

2.4. state machine Cov

The coverage group is defined as below in the file memsys_coverage.v
 This also can be directly defined inside rtl module cntlr.

```

enum logic [1:0] {IDLE, START, WRITE0, WRITE1} st;
covergroup cntlr_cov @vintf.CBmemsys;
    b: coverpoint memsys_test_top.dut.Umem.state {
        bins t0 = (IDLE => IDLE);
        bins t1 = (IDLE => START);
        bins t2 = (START => IDLE);
        bins t3 = (START => WRITE0);
        bins t4 = (WRITE0 => WRITE1);
        bins t5 = (WRITE1 => IDLE);
        bins bad_trans = default;
    }

```

endgroup

HVP Measure snps	
Source	group: memsys_test_top.testbench::cntl*
Group	

-- Below we instantiate 4 sub plans for 4 srams and we pass the root_mod attribute appropriately. Check the sub plan for the spec of the sram. The sub plan is included as shown in the section “D Include File” of page 10 of this document --

3. memory0

3.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u0.")

4. memory1

4.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u1.")

5. memory2

5.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u2.")

6. memory3

6.1. subplan sram_plan #(root_mod="memsys_test_top.dut.u3.")

A.Metric Definition

--- This section is where user defined metrics are defined. It is optional and needed only when the user requires metrics outside of the VMM Planner built-in metrics---

HVP metric <code>bugs</code>	
Type	<code>Integer</code>
Aggregator	<code>Sum</code>
Goal	<code>bugs < 3</code>

B.Attribute Definition

---Here is where user defined attributes are listed. This is optional---

HVP attribute <code>phase</code>	
Type	<code>Integer</code>
Default	<code>1</code>

HVP attribute <code>version</code>	
Type	<code>Real</code>
Default	<code>0.5</code>

C.Annotation Definition

--- user defined annotations are like attributes except that they are not hierarchically inherited. This section is optional ---

HVP annotation <code>spec</code>	
Type	<code>String</code>
Default	

D.Include File

HVP include file	
<code>sram_wordplan.xml</code>	